CompLit

LEARN. PRACTICE. BUILD. SUCCEED.

CompLit System Requirements Document

04/20/2026

Team: Origami Inc.

Members: Ronan Fowler, Cedrick Gutierrez, and Matthew Juhls

Table of Contents

Introduction (M.J.)

The purpose of this document is to show all the background processes within our app. This document is all-encompassing, so each section of the app will be covered. Included will be a model of the basic functions of input and output, the processes in each section, and the security our app will have. For the diagrams included, there will be a class diagram to show all the methods needed for the development process, a use case diagram, a use case scenarios chart to detail how each function will work, and a system sequence diagram to show the order in which the functions will operate.

Description Model

1. Output (R.F.)

The requirements for the output of our system are as follows:

1. Servers to run our program's lessons, simulations, and data on
2. An AI system to generate recommendations and feedback for users
3. The system will output a personalized user dashboard that displays progress tracking, completed lessons, badges, streaks, weak topics, and recommended next steps.

2. Input (R.F.)

The requirements for the input of our system are as follows:

1. Users who have access to the internet through either a mobile device or PC (preferred)
2. A website where users can access the download for our application which is then used to access the cloud component of it
3. A database containing login/account information of users in order for them to properly access our application
4. Moderators to review what content is uploaded to our systems
5. Content creators (optional) to input additional content to the system

3. Processes (M.J.)

The Processes that will go on in CompLit go as follows:

1. **Sign In**
   Make sure the user is signed in to figure out what classes a person is enrolled in, how many lessons they've completed, and which ones need to be started or finished, and determine how many badges the user has earned from minigames. Sign-in can be completed by inputting credentials or using a Google account. Users must sign in to use the app. If not signed in, the system will prompt the user to sign in or create an account. If the user decides to create an account instead of signing in using a Google account, then they must create a unique username with a password. Passwords must be strong and include letters, numbers, and at least 1 symbol.
2. **Text Lesson Moderation**
   AI Auto moderation systems will make sure the text lessons uploaded to the app are one, for the specified age rating and its corresponding level of difficulty, but two, make sure the information contained in the lesson is accurate to ensure the user is being taught correct skills. Auto-moderation will rely on an AI software to scan each lesson and determine the age/difficulty level, and give it an official rating. This would be happening live and would not require updates or downtime.
3. **Community Forum**
   Pull from uploaded and verified lessons to publish to the community forum. Find the details of the user who published the lesson to give credit to and allow users to view more of their lessons. Verified lessons come from verified users. Verified users gain their verification via an algorithm that takes into account the number of plays and views their lessons have, as well as the lack of auto-moderation needed for uploaded lessons. This allows the app to determine whether or not the user is uploading accurate lessons with useful knowledge that many users are seeking. Users will also be able to search and filter content in the community forum to find exactly what they want to learn.
4. **Minigames Questions**
   Unless question topics are specifically chosen by the user, the random question option will use an AI algorithm to randomly pull from or generate new questions based on official lessons the user has completed. These questions that are selected have already been run through auto-moderation and featured in posted text lessons. Each question in a lesson will contain the answers, so when a user answers a lesson in a minigame, they will see the answer and the justification.

The number of questions will also rely on the duration of the game. If the user plays a short game, then there won't be as many questions, but if the user plays a continuous game, there will be lots of questions. The goal is to have a large catalog of fun games that are made to be slightly more educational.

5. **Virtual Machine/PC Builder**
   Upon choosing a lesson in the interactive virtual environment or virtual PC builder, the virtual machine will initialize and set the user up on the home screen or the pc builder as needed. The user will need to be using a device strong enough to support the VM, and before launch, stress tests of the VM will be conducted to see what device is the minimum needed.
6. **Badge System**
   After the completion of a lesson of any type or minigame, a corresponding badge will be awarded to the user. Users will have a section on their profile where they're able to view all of their obtained badges, as well as an option to display 3 of their favorite badges. Only official content will have the ability to award badges. Official content will be created both by CompLit and outsourced to reputable textbook companies. Upon launch, there should be plenty of content for users to consume, allowing them not to have to rely on the community forum. Going forward, CompLit will also continue to create and outsource new lessons as the community does the same.
7. **Payment System**
   A scheduled monthly payout will be sent to community contributors who accrue a certain number of plays or views on their uploaded lessons. The payout amount requires a minimum amount of plays and views to allow for eligibility and payout scales as the amount of plays and views increases. Upon initial launch, these payouts will be smaller, but the required views/plays will also be lower. As the app grows in popularity and user base, the payouts will increase, and so will the minimum required views/plays. Although there is a scale worked out, in order to find an exact dollar amount, data on how many users there are is needed first.
8. **Feedback**
   An AI system to give feedback to the user in each section of the app. Feedback can be on text lessons with interactive questions for the user to answer, the VM sections where users are getting real-time feedback from the AI, and in minigames after a question is answered.

4. Performance Requirements (C.G.)

The CompLit system must meet these performance expectations:

1. **Lesson loading**
   Text and video lessons should open and be usable within about 3 seconds on a minimum connection of 100/20 **Mbps** download/upload.
2. **Status and progress updates**
   Changes like marking lessons read/completed, saving bookmarks, video lesson watch progression, and updating progress dashboards (skills learned, streaks, weak areas, badges earned) should show in the interface within 1 second.
3. **Practice activity responsiveness**
   Minigames and practice activities such as term–definition matching, hardware identification, timed challenges, and scenario-based challenges should react to user actions (answers, hints, next question) within about 500 ms so they feel smooth.
4. **Simulation startup**
   Virtual environments and computer-building simulations should start within about 5 seconds under the max server load conditions.
5. **Feedback and recommendations**
   Real-time feedback (correct/incorrect, hints) and follow-up suggestions based on performance should appear within 3 seconds of the user's action.
6. **Dashboard performance**
   User dashboards (points, badges, streaks, weak topics) should load within 3 seconds for typical users.
7. **Concurrent users**
   The system should support at least 2,500 concurrent learners using lessons, activities and simulations at launch without noticeable slowdown and we will increase our server capacity as our user count continues to grow.
8. **Video playback**
   Videos should begin playing within about 2 seconds after pressing play, and changing speed or subtitles should take effect within 1 second.

9. **Library search**
   Searching and filtering options such as lesson type, minigames/practice activity, and weak areas for example in the content/community library should return results within 3 seconds, even as the library grows.
10. **UI theme and System Context**

    UI themes to include child, teen, adult, light, dark, sepia, and high contrast. Theme changes must be applied throughout the interface within 1 second. Age-based themes shall also adjust system wording, reading complexity, visual emphasis, and instructional style to match the selected age group, while accessibility-oriented themes shall preserve readable contrast, clear focus states, and usable navigation

5. Security Requirements (C.G.)

The system must meet the following security and privacy standards to protect learner data, educator content, and community submissions:

1. **Secure accounts and sessions**
   User passwords and session tokens must be stored securely (hashed/encrypted) and never shown or logged in plain text.
2. **Strong login and MFA**
   Login must follow best practices such as 12 minimum character length, mix of uppercase letters, lowercase letters, numbers, and symbols (protected passwords, limited retries). Multi-factor authentication should be available for higher-privilege roles like creators, moderators and admins.
3. **Role-based access**
   Permissions must be based on roles (learner, educator, moderator, admin, creator). Users can only view or change their own profile, progress, and content.
4. **Protection of learner data**
   Progress, scores, and weak topics must be treated as sensitive data. Only the learner and authorized roles may see detailed performance information.
5. **Secure community content and payouts**
   Only authorized staff may approve, reject, or edit *CommunityContent* and its

moderation notes. Payout and usage information must be restricted to appropriate roles.

6. **Encryption in transit and at rest**
   All traffic between app and server must use HTTPS. Sensitive data stored in databases must be encrypted where appropriate.

7. **Safe handling of user-generated text**
   All text submitted by users (titles, descriptions, comments, etc.) must be cleaned or encoded before display to prevent script injection or harmful content.
8. **Logging and monitoring**
   Important actions (logins, role changes, approvals, payouts, account locks) must be logged so security problems can be detected and investigated.
9. **Secure third-party services**
   Any external services (text-to-speech, video hosting, payments, etc.) must use secure APIs, share only minimal necessary data, and keep keys/tokens hidden from end users.

Class Diagram (C.G.)

Our class diagram is linked in GitHub.

Use Case Diagram (R.F.)

Our use case diagram is linked in GitHub.

Use Case Detailed Descriptions (R.F.)

Our use case detailed descriptions document is linked in GitHub.

System Sequence Diagram (M.J.)

Our system sequence diagram is linked on GitHub.